Data Engineering Task

Contents

Introduction to Lead Model 1
Introduction to Task 1
Extract & Load 2
Transformations - Python 3
Python Project Guidelines 3

Introduction to Lead Model

To complete the task, it is essential to understand the concept of a lead and its life cycle. Clients seeking health support contact us for assistance. During a conversation between our employee and the client, the client identifies a need for a specific product (i.e., wheelchair, 24-hour care, etc.). Once this need is confirmed, we forward the client's contact details as a lead to companies that supply the required product.

The supplier company can take one of two actions regarding the lead:

1- **Accept the lead**: Proceed with the provided client information.
2- **Request a cancellation**: This may happen for various reasons, such as being unable to reach the client.

When a cancellation is requested, our employee reviews the request and determines the outcome:

1- If the request is **accepted**, the lead is canceled.
2- If the request is **rejected**, the lead remains active.

During its lifecycle, a lead can have four states:

- Sold
- Cancellation Requested
- Cancellation Confirmed
- Cancellation Rejected

Introduction to Task

Before calculating the events of the lead, follow these steps:

1. Create the SQL and Snowflake resources.
2. Create a data pipeline using Azure Data Factory to process the provided Excel data (100 leads):

- Source: SQL
- Destination: Snowflake

In this step, the SQL table is moved to Snowflake (no transformations).

3. Transform the data using Python.
4. Save the transformed data into a new table in Snowflake.

Extract & Load

Step 1: SQL

The Excel data must first be imported into a SQL table. Provide SQL scripts to:

1. Create the database
2. Create the table (Leads)
3. Insert Excel data into SQL

Excel Data Columns

Column Name	Type	Definition
Id	UUID	Unique identifier for each lead or row
State	Int	The current state of the lead (0 to 3)
CreatedDateUtc	DateTime	Sold Date (State = 0)
CancellationRequestDateUtc	DateTime	Date cancellation was requested (State = 1)
CancellationDateUtc	DateTime	Date cancellation was confirmed (State = 2)
CancellationRejectionDateUtc	DateTime	Date cancellation was rejected (State = 3)
SoldEmployee	String	Email of the employee who sold the lead
CancelledEmployee	String	Email of the employee who canceled the lead
UpdatedDateUtc	DateTime	Last DateTime the row was updated

Step 2: Azure Data Factory Pipeline

Create a pipeline to move data from SQL to Snowflake, ensuring:

- Incremental loads
- GitHub integration
- Extensibility and maintainability
- Email alerts for failures
- Usage of linked services
- A trigger that runs every 30 minutes

Step 3: Snowflake

Provide SQL scripts to set up the Snowflake instance with the same SQL schema. Provide scripts to:

1. Create a Warehouse instance
2. Create the database
3. Create the tables (Leads)

Transformations - Python

The task involves:

1. Read data from the Leads table in Snowflake.
2. Perform transformations to calculate events in the lead's life cycle.
3. Store the transformed data in the LeadEvents table in Snowflake.

Transformed Table Structure

Column Name	Data Type	Definition
Id	UUID	Unique identifier for each event row
EventType	String	Type of event (e.g., LeadSold, LeadCancelled, etc.)
EventEmployee	String	Email of the person who performed the event
EventDate	DateTime	Date and time of the event
LeadId	UUID	Unique identifier of the lead
UpdatedDateUtc	DateTime	Last updated DateTime during event calculation

Column Logic:

- **EventType**:
  - LeadSold (State = 0)
  - LeadCancellationRequested (State = 1)
  - LeadCancelled (State = 2)
  - LeadCancellationRejected (State = 3)
- **EventEmployee**:
  - For Sold events, use **SoldEmployee**
  - For Cancellation events, use **CancelledEmployee**
  - For **LeadCancellationRequested** and **LeadCancellationRejected**, use "Unknown".
- **EventDate**:
  - Corresponds to the appropriate date column (e.g., CreatedDateUtc).

Python Project Guidelines

- Use a clean project and folder structure.
- Follow consistent naming conventions.
- Include meaningful comments and proper exception handling.
- Avoid hardcoding secrets in the code.
- Apply object-oriented principles and design patterns for external service connections.
- Ensure the code is clean, efficient, and adaptable to changes.

Best of luck with the task! 🎉